Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing (CS426L)

Lab Report 05: Synchronization Mechanisms

C# Edition (.NET, System.Threading)

Student Name	Fahad Noor
Registration No.	232448
Section	BSCS-ev-7-C
Instructor	Muhammad Nauman Zafar Hashmi
Date	7 October 2026

Section 1: Environment Setup

Confirmed with dotnet --version that the .NET SDK carried over from Labs 02 to 04 is still available. A working folder for Lab 05 was created containing five console projects: BufferMutex, BufferSemaphore, CondSignal, PhaseBarrier and LockFreeStack. Every program was built with dotnet build -c Release and run with dotnet run --no-build -c Release.

Number of logical cores on this machine (Environment.ProcessorCount): 8

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\LockFreeStack>dotnet --version
10.0.401
```

Section 2: Task 1 — Mutex-Protected Bounded Buffer

Source file: BufferMutex/Program.cs

```
// Lab 05 - Task 1: Mutex-Protected Bounded Buffer
// One producer and one consumer share a circular buffer protected by a lock.
// When the buffer is full (or empty) the thread leaves the lock, counts a
// retry and calls Thread.Yield() before trying again (polling).
//
// Run:  dotnet run --no-build -c Release          (buffer size 8)
//       dotnet run --no-build -c Release -- 1     (buffer size 1)
//       dotnet run --no-build -c Release -- 64    (buffer size 64)

using System;
using System.Diagnostics;
using System.Threading;

class Program
{
    static int BufferSize = 8;              // capacity of the bounded buffer
    const int NumItems = 1_000_000;         // items sent from producer to consumer

    // ---- shared state (protected by gate) ----
    static int[] buffer = new int[BufferSize];
    static int inPos = 0;                   // next slot to write
    static int outPos = 0;                  // next slot to read
    static int count = 0;                   // items currently in the buffer
    static readonly object gate = new object();

    // ---- statistics (each written by one thread only, read after Join) ----
    static long producerRetries = 0;
    static long consumerRetries = 0;
    static long checksumProduced = 0;
    static long checksumConsumed = 0;
    static long orderErrors = 0;

    static void Producer()
    {
        for (int item = 1; item <= NumItems; item++)
        {
            checksumProduced += item;

            while (true)
            {
                bool inserted = false;

                lock (gate)
                {
                    // test and insert in the SAME critical section
                    if (count < BufferSize)
                    {
                        buffer[inPos] = item;
                        inPos = (inPos + 1) % BufferSize;
                        count++;
                        inserted = true;
                    }
                }

                if (inserted) break;        // done with this item

                producerRetries++;          // buffer was full: try again later
```

```
            Thread.Yield();
        }
    }
}

static void Consumer()
{
    for (int expected = 1; expected <= NumItems; expected++)
    {
        int item = 0;

        while (true)
        {
            bool removed = false;

            lock (gate)
            {
                // test and remove in the SAME critical section
                if (count > 0)
                {
                    item = buffer[outPos];
                    outPos = (outPos + 1) % BufferSize;
                    count--;
                    removed = true;
                }
            }

            if (removed) break;

            consumerRetries++;          // buffer was empty: try again later
            Thread.Yield();
        }

        // verification: items must arrive as 1, 2, 3, ...
        checksumConsumed += item;
        if (item != expected) orderErrors++;
    }
}

static void Main(string[] args)
{
    if (args.Length > 0 && int.TryParse(args[0], out int size) && size > 0)
    {
        BufferSize = size;
        buffer = new int[BufferSize];
    }

    Thread producer = new Thread(Producer);
    Thread consumer = new Thread(Consumer);

    Stopwatch sw = Stopwatch.StartNew();
    producer.Start();
    consumer.Start();
    producer.Join();
    consumer.Join();
    sw.Stop();

    bool checksumOk = checksumProduced == checksumConsumed;

    Console.WriteLine($"Buffer size: {BufferSize}");
    Console.WriteLine($"Items: {NumItems}");
    Console.WriteLine($"Checksum produced: {checksumProduced}");
    Console.WriteLine($"Checksum consumed: {checksumConsumed}");
    Console.WriteLine($"Checksum OK: {(checksumOk ? "Y" : "N")}");
    Console.WriteLine($"Order errors: {orderErrors}");
    Console.WriteLine($"Producer retries: {producerRetries}");
    Console.WriteLine($"Consumer retries: {consumerRetries}");
    Console.WriteLine($"Time: {sw.Elapsed.TotalMilliseconds:F2} ms");
}
}
```

Run results (buffer size 8)

Run	Items	Checksum OK (Y/N)	Order errors	Producer retries	Consumer retries	Time (ms)
1	1,000,000	Y	0	126,216	121,735	84.63
2	1,000,000	Y	0	124,367	124,836	104.42
3	1,000,000	Y	0	123,300	123,393	395.27
4	1,000,000	Y	0	127,319	124,542	423.42
5	1,000,000	Y	0	143,557	1,852,585	2587.68
Average	1,000,000	Y (5/5)	0	128,951.8	469,418.2	719.08

Effect of buffer size on retries (one run each)

Buffer size	Producer retries	Consumer retries	Time (ms)
1	1,251,930	1,255,209	2931.27
8	126,216	121,735	84.63
64	21,663	16,955	189.26

Buffer size and number of items used for the first table: buffer size 8, 1,000,000 items

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferMutex>dotnet run --no-build -c Release
Buffer size: 8
Items: 1000000
Checksum produced: 500000500000
Checksum consumed: 500000500000
Checksum OK: Y
Order errors: 0
Producer retries: 126216
Consumer retries: 121735
Time: 84.63 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferMutex>dotnet run --no-build -c Release
Buffer size: 8
Items: 1000000
Checksum produced: 500000500000
Checksum consumed: 500000500000
Checksum OK: Y
Order errors: 0
Producer retries: 124367
Consumer retries: 124836
Time: 104.42 ms
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferMutex>dotnet run --no-build -c Release -- 1
Buffer size: 1
Items: 1000000
Checksum produced: 500000500000
Checksum consumed: 500000500000
Checksum OK: Y
Order errors: 0
Producer retries: 1251930
Consumer retries: 1255209
Time: 2931.27 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferMutex>dotnet run --no-build -c Release -- 64
Buffer size: 64
Items: 1000000
Checksum produced: 500000500000
Checksum consumed: 500000500000
Checksum OK: Y
Order errors: 0
Producer retries: 21663
Consumer retries: 16955
Time: 189.26 ms
```

Discussion Questions

1. Was every item received in order, and did the checksum always match? Explain why a single producer and a single consumer working on a circular buffer preserve first-in, first-out order.

Yes. In all 5 runs (and in the buffer size 1 and 64 runs) the consumed checksum equalled the produced checksum (500,000,500,000) and there were 0 order errors.

Order is preserved because there is exactly one writer of inPos and one writer of outPos. The producer puts items into slots inPos, inPos+1, ... in the order 1, 2, 3, ..., and the consumer takes them out from outPos, outPos+1, ... in the same circular order. Both indexes move in the same direction around the ring, and count stops the producer from overwriting a slot that has not been read and stops the consumer from reading a slot that has not been written. So the first item in is always the first item out.

2. Why must the test of count and the insertion happen inside the same critical section? Describe what could go wrong if the producer tested count < BufferSize first, released the lock, and locked again to insert.

The decision ("there is room") and the action (insert and count++) must be one atomic step, otherwise the decision can be out of date by the time the action happens. This is a check-then-act race.

If the producer tested count < BufferSize, released the lock and locked again to insert, another thread could change the buffer in between. With two producers, both could see count = 7 with BufferSize = 8, both decide there is room, and both insert: the second insert overwrites an unread item and count becomes 9, so an item is lost and the buffer state is corrupted. Even with one producer the code would be relying on luck rather than on the lock.

3. Compare the retry counts for buffer sizes 1, 8 and 64. What do the retries represent in terms of CPU time, and why does Thread.Yield() reduce but not remove the waste?

Buffer size 1: 1,251,930 producer and 1,255,209 consumer retries (2931.27 ms). Buffer size 8: 126,216 and 121,735 retries (84.63 ms). Buffer size 64: 21,663 and 16,955 retries (189.26 ms). Retries fall sharply as the buffer grows: about 10 times fewer from size 1 to 8, and about 6 to 7 times fewer again from 8 to 64. With size 1 there was more than one retry for every item on each side.

Each retry is a wasted trip through the lock: the thread acquired the lock, found the buffer full (or empty), released it and did no useful work. That is CPU time spent polling. A larger buffer lets one thread do many operations before it has to wait for the other, so it runs into the full/empty condition less often.

Thread.Yield() only offers the core to another thread that is ready to run. On this 8-core machine the other thread already has its own core, so Yield usually returns immediately and the thread polls again. It reduces the waste a little but the thread is still runnable and still burning CPU; it never sleeps until the condition actually changes. The timings also show how unstable polling is: the five buffer-size-8 runs ranged from 84.63 ms to 2587.68 ms, and in run 5 the consumer retried 1,852,585 times.

4. Suppose the producer, on finding the buffer full, simply slept for a while while still holding the lock. What would happen to the consumer, and why?

The consumer would be blocked at lock (gate) for the whole time the producer sleeps, because the producer still owns the lock. The consumer is the only thread that can make space in the buffer, but it cannot get in to remove an item. So the producer would wake up, find the buffer still full, and sleep again. If it never releases the lock while waiting, the program makes no progress at all (a deadlock/livelock: the producer waits for the consumer, and the consumer waits for the lock the producer holds).

This is why the code leaves the lock before calling Thread.Yield(), and why a proper wait (Monitor.Wait, used in Task 3) releases the lock while it sleeps.

Section 3: Task 2 Multiple Producers and Consumers with Semaphores

Source file: BufferSemaphore/Program.cs

```
// Lab 05 - Task 2: Multiple Producers and Consumers with Semaphores
// emptySlots counts free slots (starts at capacity), fullSlots counts items
// (starts at 0). A lock still protects the indexes of the circular buffer.
// Protocol:  Wait() -> lock (gate) -> update index -> leave lock -> Release()
//
// Run:  dotnet run --no-build -c Release -- <producers> <consumers> <capacity>
//       e.g.  dotnet run --no-build -c Release -- 2 2 8
//
// Deadlock experiment (Question 3):
//       dotnet run --no-build -c Release -- 1 1 8 deadlock
//       (consumer takes the lock BEFORE fullSlots.Wait(); stop with Ctrl+C)

using System;
using System.Diagnostics;
using System.Threading;

class Program
{
   const int TotalItems = 400_000;            // divisible by 1, 2 and 4

   static int numProducers = 2;
   static int numConsumers = 2;
   static int bufferSize = 8;
   static bool deadlockExperiment = false;

   // ---- shared state ----
   static int[] buffer = Array.Empty<int>();
   static int inPos = 0;
   static int outPos = 0;
   static readonly object gate = new object();

   static SemaphoreSlim emptySlots = null!;   // free slots in the buffer
   static SemaphoreSlim fullSlots = null!;    // items waiting in the buffer

   static int[] seen = Array.Empty<int>();    // seen[id] = how many times item id was consumed

   static void Producer(int id, int itemsToProduce)
   {
      int firstId = id * itemsToProduce;     // every producer has its own id range

      for (int i = 0; i < itemsToProduce; i++)
      {
         int item = firstId + i;

         emptySlots.Wait();                  // 1. wait for a free slot (blocks if full)

         lock (gate)                         // 2. protect the index update
         {
            buffer[inPos] = item;
            inPos = (inPos + 1) % bufferSize;
         }

         fullSlots.Release();                // 3. tell consumers one item is ready
      }
   }

   static void Consumer(int itemsToConsume)
   {
      for (int i = 0; i < itemsToConsume; i++)
      {
         int item;

         if (!deadlockExperiment)
         {
            fullSlots.Wait();                // 1. wait for an item (blocks if empty)

            lock (gate)                      // 2. protect the index update
            {
               item = buffer[outPos];
               outPos = (outPos + 1) % bufferSize;
```

```
            }
        }
        else
        {
            // WRONG ORDER (experiment only): lock first, then wait.
            // The consumer sleeps on the semaphore while holding the lock,
            // so the producer can never get in to add an item.
            lock (gate)
            {
                fullSlots.Wait();
                item = buffer[outPos];
                outPos = (outPos + 1) % bufferSize;
            }
        }

        emptySlots.Release();              // 3. tell producers one slot is free

        Interlocked.Increment(ref seen[item]);
    }
}

static void Main(string[] args)
{
    if (args.Length >= 3)
    {
        numProducers = int.Parse(args[0]);
        numConsumers = int.Parse(args[1]);
        bufferSize = int.Parse(args[2]);
    }
    deadlockExperiment = args.Length >= 4 && args[3] == "deadlock";

    if (TotalItems % numProducers != 0 || TotalItems % numConsumers != 0)
    {
        Console.WriteLine($"Producers and consumers must divide {TotalItems} evenly.");
        return;
    }

    buffer = new int[bufferSize];
    seen = new int[TotalItems];
    emptySlots = new SemaphoreSlim(bufferSize, bufferSize);   // all slots free
    fullSlots = new SemaphoreSlim(0, bufferSize);             // no items yet

    int perProducer = TotalItems / numProducers;
    int perConsumer = TotalItems / numConsumers;

    Thread[] producers = new Thread[numProducers];
    Thread[] consumers = new Thread[numConsumers];

    if (deadlockExperiment)
        Console.WriteLine("Deadlock experiment: consumer locks BEFORE fullSlots.Wait(). Press Ctrl+C to stop.");

    Stopwatch sw = Stopwatch.StartNew();

    for (int c = 0; c < numConsumers; c++)
    {
        consumers[c] = new Thread(() => Consumer(perConsumer));
        consumers[c].Start();
    }
    for (int p = 0; p < numProducers; p++)
    {
        int id = p;                        // private copy for the lambda
        producers[p] = new Thread(() => Producer(id, perProducer));
        producers[p].Start();
    }

    foreach (Thread t in producers) t.Join();
    foreach (Thread t in consumers) t.Join();
    sw.Stop();

    // verification: every item id must have been consumed exactly once
    int lost = 0, duplicated = 0;
    for (int i = 0; i < TotalItems; i++)
    {
        if (seen[i] == 0) lost++;
        else if (seen[i] > 1) duplicated += seen[i] - 1;
    }
```

```
        Console.WriteLine($"Config: producers={numProducers} consumers={numConsumers} capacity={bufferSize}");
        Console.WriteLine($"Items: {TotalItems}");
        Console.WriteLine($"Lost: {lost}");
        Console.WriteLine($"Duplicated: {duplicated}");
        Console.WriteLine($"Time: {sw.Elapsed.TotalMilliseconds:F2} ms");
    }
}
```

Configurations (three runs each)

Producers / consumers / capacity	Lost	Duplicated	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)
1 / 1 / 8	0	0	714.78	62.19	143.55	306.84
2 / 2 / 8	0	0	261.38	190.81	207.52	219.90
4 / 4 / 8	0	0	375.77	346.56	358.30	360.21
4 / 4 / 1	0	0	739.85	902.00	944.88	862.24
4 / 4 / 64	0	0	193.63	172.11	162.63	176.12
4 / 2 / 8	0	0	311.10	315.59	303.11	309.93

Total items per run: 400,000 (shared equally between the producers).

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferSemaphore>dotnet run --no-build -c Release -- 1 1 8
Config: producers=1 consumers=1 capacity=8
Items: 400000
Lost: 0
Duplicated: 0
Time: 714.78 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferSemaphore>dotnet run --no-build -c Release -- 2 2 8
Config: producers=2 consumers=2 capacity=8
Items: 400000
Lost: 0
Duplicated: 0
Time: 261.38 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferSemaphore>dotnet run --no-build -c Release -- 4 4 8
Config: producers=4 consumers=4 capacity=8
Items: 400000
Lost: 0
Duplicated: 0
Time: 375.77 ms
```

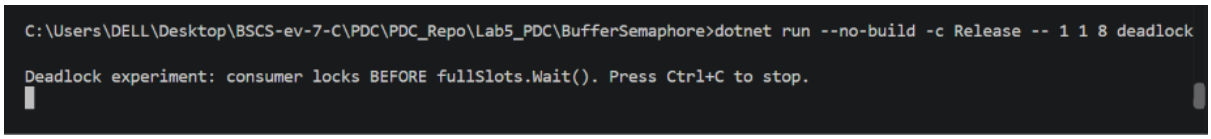
```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferSemaphore>dotnet run --no-build -c Release -- 4 4 8
Config: producers=4 consumers=4 capacity=8
Items: 400000
Lost: 0
Duplicated: 0
Time: 375.77 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferSemaphore>dotnet run --no-build -c Release -- 4 4 1
Config: producers=4 consumers=4 capacity=1
Items: 400000
Lost: 0
Duplicated: 0
Time: 739.85 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\BufferSemaphore>dotnet run --no-build -c Release -- 4 4 64
Config: producers=4 consumers=4 capacity=64
Items: 400000
Lost: 0
Duplicated: 0
Time: 193.63 ms
```

Deadlock experiment (consumer locks before fullSlots.Wait())

The program was run as dotnet run --no-build -c Release -- 1 1 8 deadlock, which makes the consumer enter lock (gate) before calling fullSlots.Wait(). It printed only the line "Deadlock experiment: consumer locks BEFORE fullSlots.Wait(). Press Ctrl+C to stop." and then produced no further output: the Lost / Duplicated / Time lines never appeared. The program hung and had to be stopped with Ctrl+C. The correct order (Wait first, then lock) was used for all recorded runs.



Discussion Questions

1. Explain the roles of the two semaphores and why their initial values are the buffer capacity and 0.

emptySlots counts how many free slots the buffer has, and fullSlots counts how many items are waiting in it. A producer calls emptySlots.Wait() before inserting, so it blocks when the buffer is full, and calls fullSlots.Release() afterwards to announce a new item. A consumer calls fullSlots.Wait() before removing, so it blocks when the buffer is empty, and calls emptySlots.Release() afterwards to announce a free slot.

At the start the buffer is empty: all slots are free and there are no items. So emptySlots starts at the capacity and fullSlots starts at 0. At every moment emptySlots + fullSlots (plus operations in progress) equals the capacity.

2. Why is a mutex still needed in addition to the semaphores once there are several producers and several consumers? Would it also be needed with exactly one producer and one consumer? Justify your answer.

The semaphores only control how many threads may proceed; they do not protect the indexes. With capacity 8, up to 8 producers could pass emptySlots.Wait() at the same time, and all of them would then read and update inPos together. Two producers could write to the same slot and both advance inPos, losing one item and leaving a slot with stale data (a duplicate). The same applies to consumers and outPos. The lock makes the read-update of the index and the buffer access a critical section.

With exactly one producer and one consumer the lock is not strictly needed: only the producer touches inPos and only the consumer touches outPos, and the semaphores guarantee they are never working on the same slot (a slot is written before fullSlots is released and read before emptySlots is released). The semaphore operations also provide the memory visibility. We still keep the lock so that the same code is correct for any number of threads.

3. In the deadlock experiment, describe step by step why the program stops making progress when a consumer takes the mutex before waiting on the full-slot semaphore.

1. The buffer is empty (at the start, or whenever the consumer catches up). The consumer enters lock (gate) and now owns the lock.

2. Still holding the lock, it calls fullSlots.Wait(). The count is 0, so the consumer goes to sleep. A semaphore wait does not release the lock.

3. The producer passes emptySlots.Wait() (there are free slots) and then tries to enter lock (gate) to insert its item. The lock is held by the sleeping consumer, so the producer blocks.

4. Now the consumer is waiting for an item that only the producer can supply, and the producer is waiting for the lock that only the consumer can release. Neither can ever continue: a deadlock. In the experiment the program printed its first line and then hung until Ctrl+C. Waiting on the semaphore first and taking the lock only for the short index update avoids this, because no thread ever sleeps while holding the lock.

4. How did the time change with the buffer capacity (1, 8, 64) and with the number of threads? Explain your observations in terms of blocking and contention.

Capacity (4 producers, 4 consumers): 739.85 ms with capacity 1, 375.77 ms with capacity 8 and 193.63 ms with capacity 64. A larger buffer is faster. With capacity 1 every single item forces a hand-over: a producer can insert one item and then must block until a consumer removes it, so threads are constantly put to sleep and woken up, which is expensive. With capacity 64 producers and consumers can each do many operations before they have to block, so there are far fewer sleeps and wake-ups.

Number of threads (capacity 8): 714.78 ms for 1/1, 261.38 ms for 2/2, 375.77 ms for 4/4 and 311.10 ms for 4/2. Going from 1/1 to 2/2 was faster, probably because with more threads there is more often a thread already awake to take the next item, so fewer operations have to wait for a wake-up. Going to 4/4 was slower again: all 8 threads compete for the same lock and the same two semaphores, so contention grows while the work per item is tiny and cannot really be done in parallel.

Each configuration was run only once here, so these comparisons are indicative; the trend with capacity is large and clear, while the differences between thread counts are within what run-to-run noise could produce.

5. Compare the 1 / 1 / 8 semaphore version with the Task 1 polling version (time and CPU use). Why does blocking differ from polling?

The 1/1/8 semaphore version took 714.78 ms for 400,000 items (about 1.8 microseconds per item). The Task 1 polling version handled 1,000,000 items in 84.63 to 2587.68 ms (average 719.08 ms), so in most runs polling was faster per item in wall-clock time.

The difference is in what the threads do while they wait. In the polling version a waiting thread stays runnable: it keeps taking the lock, checking count and retrying (about 125,000 retries per side per run), so both threads use a full core for the whole run. In the semaphore version a thread that cannot proceed is put to sleep by Wait() and uses no CPU until Release() wakes it. Blocking and waking a thread costs more per hand-over than a quick retry, which is why the wall time is not better here, but the CPU is free for other work while a thread waits. CPU time was not measured for these two programs; Task 3 (scenario A versus D) measures this difference directly.

Polling is only attractive when waits are extremely short and spare cores are available; blocking is the right choice when waits can be long or cores are shared.

6. Is first-in, first-out order guaranteed as seen by multiple consumers? Explain why the seen array checks for lost and duplicated items instead of checking order.

No. Items leave the buffer in FIFO order, but with several consumers the order in which they are processed is not defined: consumer A may remove item 10 and be preempted before recording it, while consumer B removes and records item 11. With several producers there is not even a single production order, because the producers interleave their inserts.

So an order check would report errors for a perfectly correct program. What must hold in every configuration is that each item is delivered exactly once. The seen array counts how many times each item id was consumed: 0 means the item was lost and more than 1 means it was duplicated. All six configurations reported Lost: 0 and Duplicated: 0.

Section 4: Task 3 Condition-Variable Signaling

Source file: CondSignal/Program.cs

```
// Lab 05 - Task 3: Condition-Variable Signaling
// A worker blocks on a condition (Monitor.Wait) until another thread sets the
// "data ready" flag and signals it (Monitor.Pulse / Monitor.PulseAll).
//
// Scenarios (dotnet run --no-build -c Release -- <arguments>):
//   A:  cond                one worker waits, then main publishes
//   B:  cond 1 signal-first   main publishes BEFORE the worker starts
//   C:  cond 3              three workers wait for one publish
//   D:  spin                one worker polls the flag in a busy loop
//
```

```
// Lost-signal experiment (Question 3):
//       cond 1 no-signal      flag is set but nobody is woken (stop with Ctrl+C)

using System;
using System.Diagnostics;
using System.Threading;

class Program
{
   const int WaitMilliseconds = 1000;         // how long main "prepares the data"
   const int PublishedValue = 42;

   // ---- shared state (protected by gate) ----
   static readonly object gate = new object();
   static bool dataReady = false;             // the predicate
   static int sharedValue = 0;

   static bool sendSignal = true;             // false only in the lost-signal experiment
   static int numWorkers = 1;

   static int[] timesBlocked = Array.Empty<int>();
   static int[] valueReceived = Array.Empty<int>();
   static long[] spinIterations = Array.Empty<long>();

   // Condition-variable version: sleeps until signaled, uses no CPU while waiting.
   static void Worker(int id)
   {
      lock (gate)
      {
         while (!dataReady)                   // re-check the predicate after every wake-up
         {
            timesBlocked[id]++;
            Monitor.Wait(gate);           // releases gate, sleeps, re-acquires gate
         }
         valueReceived[id] = sharedValue;
      }
   }

   // Polling version: keeps a core busy checking the flag.
   static void SpinWorker(int id)
   {
      long spins = 0;
      while (!Volatile.Read(ref dataReady))
      {
         spins++;
      }
      spinIterations[id] = spins;

      lock (gate)
      {
         valueReceived[id] = sharedValue;
      }
   }

   static void Publish()
   {
      lock (gate)
      {
         sharedValue = PublishedValue;
         dataReady = true;                    // set the flag first ...

         if (sendSignal)                      // ... then wake the waiting thread(s)
         {
            if (numWorkers == 1) Monitor.Pulse(gate);
            else Monitor.PulseAll(gate);
         }
      }
   }

   static void Main(string[] args)
   {
      string mode = args.Length > 0 ? args[0] : "cond";
      if (args.Length > 1) numWorkers = int.Parse(args[1]);
      string option = args.Length > 2 ? args[2] : "";

      bool signalFirst = option == "signal-first";
      sendSignal = option != "no-signal";
      bool spin = mode == "spin";
```

```
    timesBlocked = new int[numWorkers];
    valueReceived = new int[numWorkers];
    spinIterations = new long[numWorkers];

    Thread[] workers = new Thread[numWorkers];
    for (int w = 0; w < numWorkers; w++)
    {
        int id = w;                     // private copy for the lambda
        workers[w] = spin ? new Thread(() => SpinWorker(id))
                          : new Thread(() => Worker(id));
    }

    if (!sendSignal)
        Console.WriteLine("Lost-signal experiment: the flag is set but no Pulse is sent. Press Ctrl+C to stop.");

    Stopwatch sw = Stopwatch.StartNew();

    if (signalFirst)
    {
        Publish();                      // signal is sent while nobody is waiting
        foreach (Thread t in workers) t.Start();
    }
    else
    {
        foreach (Thread t in workers) t.Start();
        Thread.Sleep(WaitMilliseconds);    // workers are waiting during this time
        Publish();
    }

    foreach (Thread t in workers) t.Join();
    sw.Stop();

    double cpuMs = Process.GetCurrentProcess().TotalProcessorTime.TotalMilliseconds;

    string scenario = spin ? "spin (polling)"
                    : signalFirst ? "cond, signal-first"
                    : $"cond, {numWorkers} worker(s)";

    Console.WriteLine($"Scenario: {scenario}");
    for (int w = 0; w < numWorkers; w++)
    {
        if (spin)
            Console.WriteLine($"Worker {w}: times blocked in wait = 0, spin iterations = {spinIterations[w]}, value received =
{valueReceived[w]}");
        else
            Console.WriteLine($"Worker {w}: times blocked in wait = {timesBlocked[w]}, value received = {valueReceived[w]}");
    }
    Console.WriteLine($"Wall time: {sw.Elapsed.TotalMilliseconds:F2} ms");
    Console.WriteLine($"CPU time: {cpuMs:F2} ms");
  }
}
```

Scenario results

Scenario	Times blocked in wait	Value received	Wall time (ms)	CPU time (ms)
A: cond, 1 worker	1	42	1015.49	125.00
B: signal-first	0	42	1.48	109.38
C: cond, 3 workers	1 per worker (3 workers)	42 (all three)	1015.79	125.00
D: spin (polling)	0 (1,927,484,305 spin iterations)	42	1015.88	1046.88

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\CondSignal>dotnet run --no-build -c Release -- cond
Scenario: cond, 1 worker(s)
Worker 0: times blocked in wait = 1, value received = 42
Wall time: 1015.49 ms
CPU time: 125.00 ms
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\CondSignal>dotnet run --no-build -c Release -- cond 1 signal-first

Scenario: cond, signal-first
Worker 0: times blocked in wait = 0, value received = 42
Wall time: 1.48 ms
CPU time: 109.38 ms
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\CondSignal>dotnet run --no-build -c Release -- cond 3
Scenario: cond, 3 worker(s)
Worker 0: times blocked in wait = 1, value received = 42
Worker 1: times blocked in wait = 1, value received = 42
Worker 2: times blocked in wait = 1, value received = 42
Wall time: 1015.79 ms
CPU time: 125.00 ms
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\CondSignal>dotnet run --no-build -c Release -- spin
Scenario: spin (polling)
Worker 0: times blocked in wait = 0, spin iterations = 1927484305, value received = 42
Wall time: 1015.88 ms
CPU time: 1046.88 ms
```

Lost-signal experiment (no Monitor.Pulse in Publish())

The program was run as dotnet run --no-build -c Release -- cond 1 no-signal, which sets the flag in Publish() but skips the Monitor.Pulse call. It printed only "Lost-signal experiment: the flag is set but no Pulse is sent. Press Ctrl+C to stop." and then hung: the worker never printed its result and the program had to be stopped with Ctrl+C. The worker was already asleep in Monitor.Wait() when the flag was set, and nothing ever woke it.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\CondSignal>dotnet run --no-build -c Release -- cond 1 no-signal
Lost-signal experiment: the flag is set but no Pulse is sent. Press Ctrl+C to stop.
```

Discussion Questions

1. Why does the worker test the flag in a while loop instead of an if? Give at least two situations in which the worker must check the predicate again after waking up.

Being woken up does not guarantee that the condition is true; it only means it is worth checking again. The worker must re-check the predicate every time it wakes, and go back to waiting if it is still false. An if would check only once.

Situations where a re-check is needed: (1) Several waiters are woken (PulseAll) but only one of them can use the data; the others wake, find the condition false again, and must wait again. (2) Between the Pulse and the moment the woken thread re-acquires the lock, another thread can get the lock first and consume the data or reset the flag. (3) The pulse was sent for a different reason, when one lock object is used for more than one condition. (4) Spurious wake-ups, which condition variables in general are allowed to produce.

The while loop also handles the case where the flag is already true before the worker arrives: the loop body is simply skipped.

2. What does Monitor.Wait() do with the lock, and why must the lock be held when it is called (what exception do you get otherwise)?

Monitor.Wait(gate) atomically releases the lock and puts the thread to sleep on that object's wait queue. When it is pulsed, the thread moves to the ready queue and Wait() returns only after it has re-acquired the lock. Releasing the lock is essential: otherwise Publish() could never enter lock (gate) to set the flag.

The lock must be held because the test of the flag and the decision to sleep must be one atomic step. If the worker checked the flag without the lock, the publisher could set the flag and pulse between the check and the wait, and the worker would then sleep forever (a lost wake-up). Calling Monitor.Wait, Pulse or PulseAll without owning the lock throws SynchronizationLockException.

3. In scenario B the worker never blocks, and in the lost-signal experiment it never wakes up. Explain both observations and what they show about the role of the flag.

Scenario B: Publish() ran before the worker started, so the flag was already true. The worker entered the lock, the while (!dataReady) test was false immediately, and it never called Wait(): times blocked = 0 and wall time only 1.48 ms. The Pulse itself was sent when nobody was waiting and was simply lost, because a pulse is not remembered. The worker still got the value 42 because the flag remembered the event.

Lost-signal experiment: the flag was set but no Pulse was sent. The worker was already asleep inside Monitor.Wait(), and a sleeping thread cannot notice that a variable has changed, so it never woke up and the program hung until Ctrl+C.

Together these show that the two parts have different jobs. The flag is the state: it records that the event happened, so a thread that arrives late does not wait for a signal that has already gone. The pulse is only the wake-up call for threads that are already asleep. A correct design needs both: set the flag and pulse, under the same lock.

4. Compare the CPU time of scenario A with scenario D. Why is blocking so much cheaper than polling for the same waiting time?

Scenario A used 125.00 ms of CPU time and scenario D used 1046.88 ms, for the same wall time of about 1015 ms. Most of A's 125 ms is .NET start-up cost (scenario B, which does not wait at all, used 109.38 ms). So blocking for one second cost almost no CPU, while polling cost roughly 920 ms more, which is close to one full core for the entire wait. The spinning worker executed 1,927,484,305 loop iterations that did nothing useful.

A blocked thread is removed from the scheduler's run queue: the operating system does not schedule it, so it uses no CPU until the pulse makes it runnable again. A polling thread is always runnable and keeps a core at 100% just to read a flag that is false. That core is taken away from other threads, wastes power, and on a machine with fewer cores than threads it would actually slow down the thread that is preparing the data.

5. In scenario C, why is Monitor.Pulse() not sufficient for three waiting workers, and what does Monitor.PulseAll() change?

Monitor.Pulse() moves only one waiting thread to the ready queue. With three workers waiting, one would wake, read the value and finish, and the other two would stay asleep forever because nobody pulses again, so the program would hang.

Monitor.PulseAll() wakes every thread waiting on the object. They re-acquire the lock one at a time, each re-checks the predicate in its while loop, sees dataReady = true and reads the value. That is the right choice here because the event ("data is ready") is relevant to all waiters. In scenario C all three workers blocked once and all three received 42. Pulse() is enough only when a single waiter can make use of the event, for example one item for one consumer.

Section 5: Task 4 — Phase Barrier for Worker Threads

Source file: PhaseBarrier/Program.cs

```
// Lab 05 - Task 4: Phase Barrier for Worker Threads
// Workers update a shared grid in rounds. Every round has two phases:
//   phase 1: compute next[] from current[]   (reads neighbours' cells)
//   phase 2: copy next[] back into current[]
// A reusable barrier makes all workers finish a phase before any starts the next.
//
// Run:  dotnet run --no-build -c Release                  (4 workers, with barrier)
//       dotnet run --no-build -c Release -- --no-barrier    (4 workers, no barrier)
//       dotnet run --no-build -c Release -- 8               (8 workers, with barrier)

using System;
using System.Diagnostics;
using System.Threading;

// A reusable barrier built from a lock and a condition (Monitor.Wait / PulseAll).
```

```
class PhaseBarrier
{
    private readonly int parties;              // threads that must arrive
    private int waiting = 0;                   // threads that have arrived in this generation
    private long generation = 0;               // increases every time the barrier opens
    private readonly object gate = new object();

    public PhaseBarrier(int parties)
    {
        this.parties = parties;
    }

    public void SignalAndWait()
    {
        lock (gate)
        {
            long myGeneration = generation;    // the generation I arrived in
            waiting++;

            if (waiting == parties)
            {
                // last thread to arrive: open the barrier for everyone
                waiting = 0;
                generation++;
                Monitor.PulseAll(gate);
            }
            else
            {
                // wait until MY generation is over (not just until someone pulses)
                while (myGeneration == generation)
                {
                    Monitor.Wait(gate);
                }
            }
        }
    }
}

class Program
{
    const int Cells = 800_000;                 // divisible by 4 and 8
    const int Rounds = 100;
    const int Modulus = 1_000_003;

    static int[] current = new int[Cells];
    static int[] next = new int[Cells];

    static int numWorkers = 4;
    static bool useBarrier = true;
    static PhaseBarrier barrier = null!;

    static void Initialise(int[] grid)
    {
        for (int i = 0; i < Cells; i++)
            grid[i] = (i * 31 + 7) % 1000;
    }

    // New value of cell i in a given round: depends on the cell and both neighbours.
    static int Update(int[] grid, int i, int round)
    {
        int left = grid[(i + Cells - 1) % Cells];
        int right = grid[(i + 1) % Cells];
        return (left + 2 * grid[i] + right + round) % Modulus;
    }

    // Single-threaded reference result.
    static int[] ComputeReference()
    {
        int[] cur = new int[Cells];
        int[] nxt = new int[Cells];
        Initialise(cur);

        for (int round = 0; round < Rounds; round++)
        {
            for (int i = 0; i < Cells; i++) nxt[i] = Update(cur, i, round);
            for (int i = 0; i < Cells; i++) cur[i] = nxt[i];
        }
        return cur;
    }
```

```
    static void Worker(int id)
    {
        int start = id * (Cells / numWorkers);          // this worker's own slice
        int end = start + (Cells / numWorkers);

        for (int round = 0; round < Rounds; round++)
        {
            // phase 1: compute (reads current[], including neighbours' edge cells)
            for (int i = start; i < end; i++)
                next[i] = Update(current, i, round);

            if (useBarrier) barrier.SignalAndWait();   // everyone finished computing

            // phase 2: publish (writes current[])
            for (int i = start; i < end; i++)
                current[i] = next[i];

            if (useBarrier) barrier.SignalAndWait();   // everyone finished publishing
        }
    }

    static void Main(string[] args)
    {
        foreach (string arg in args)
        {
            if (arg == "--no-barrier") useBarrier = false;
            else if (int.TryParse(arg, out int n) && n > 0) numWorkers = n;
        }

        if (Cells % numWorkers != 0)
        {
            Console.WriteLine($"Number of workers must divide {Cells} evenly.");
            return;
        }

        int[] reference = ComputeReference();

        Initialise(current);
        barrier = new PhaseBarrier(numWorkers);
        Thread[] workers = new Thread[numWorkers];

        Stopwatch sw = Stopwatch.StartNew();
        for (int w = 0; w < numWorkers; w++)
        {
            int id = w;                     // private copy for the lambda
            workers[w] = new Thread(() => Worker(id));
            workers[w].Start();
        }
        foreach (Thread t in workers) t.Join();
        sw.Stop();

        int mismatches = 0;
        for (int i = 0; i < Cells; i++)
            if (current[i] != reference[i]) mismatches++;

        Console.WriteLine($"Mode: {(useBarrier ? "with barrier" : "no barrier")}");
        Console.WriteLine($"Workers: {numWorkers}  Cells: {Cells}  Rounds: {Rounds}");
        Console.WriteLine($"Correct: {(mismatches == 0 ? "Y" : "N")}");
        Console.WriteLine($"Mismatching cells: {mismatches}");
        Console.WriteLine($"Time: {sw.Elapsed.TotalMilliseconds:F2} ms");
        Console.WriteLine($"Environment.ProcessorCount: {Environment.ProcessorCount}");
    }
}
```

Run results (4 workers)

Run	With barrier: correct? (Y/N)	With barrier: time (ms)	No barrier: correct? (Y/N)	No barrier: mismatching cells
1	Y	278.47	N	794
2	Y	239.13	N	794
3	Y	258.15	N	791

Run	With barrier: correct? (Y/N)	With barrier: time (ms)	No barrier: correct? (Y/N)	No barrier: mismatching cells
4	Y	236.60	N	756
5	Y	174.70	N	782

Time with 8 workers (ms): 324.05 (correct: Y, 0 mismatching cells)

Average time with 4 workers (ms): 237.41

Environment.ProcessorCount: 8

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\PhaseBarrier>dotnet run --no-build -c Release
Mode: with barrier
Workers: 4  Cells: 800000  Rounds: 100
Correct: Y
Mismatching cells: 0
Time: 278.47 ms
Environment.ProcessorCount: 8

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\PhaseBarrier>
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\PhaseBarrier>dotnet run --no-build -c Release
Mode: with barrier
Workers: 4  Cells: 800000  Rounds: 100
Correct: Y
Mismatching cells: 0
Time: 239.13 ms
Environment.ProcessorCount: 8

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\PhaseBarrier>dotnet run --no-build -c Release
Mode: with barrier
Workers: 4  Cells: 800000  Rounds: 100
Correct: Y
Mismatching cells: 0
Time: 258.15 ms
Environment.ProcessorCount: 8
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\PhaseBarrier>dotnet run --no-build -c Release
Mode: with barrier
Workers: 4  Cells: 800000  Rounds: 100
Correct: Y
Mismatching cells: 0
Time: 236.60 ms
Environment.ProcessorCount: 8

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\PhaseBarrier>dotnet run --no-build -c Release
Mode: with barrier
Workers: 4  Cells: 800000  Rounds: 100
Correct: Y
Mismatching cells: 0
Time: 174.70 ms
Environment.ProcessorCount: 8

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\PhaseBarrier>dotnet run --no-build -c Release -- --no-barrier
Mode: no barrier
Workers: 4  Cells: 800000  Rounds: 100
Correct: N
Mismatching cells: 794
Time: 198.48 ms
Environment.ProcessorCount: 8
```

Discussion Questions

1. Why are two barrier calls needed in every round? What could go wrong if the second one were removed?

Each round has two phases that must not overlap: phase 1 computes next[] from current[] and reads the edge cells of the neighbouring workers' slices; phase 2 copies next[] back into current[].

The first barrier makes sure every worker has finished computing before anyone starts overwriting current[]. Without it, a fast worker would start publishing while a slower neighbour is still reading the old values.

The second barrier makes sure every worker has finished publishing before anyone starts the next round's computation. If it were removed, a fast worker would go straight into round r+1 and read a neighbour's edge cell that still holds the round r-1 value, because that neighbour has not copied its new value yet. The result would mix values from different rounds and differ from the reference, and it would differ from run to run.

2. Why does the barrier use a generation number instead of only an arrival count? Describe what could happen to a reusable barrier without it when a fast thread immediately calls the barrier again.

The barrier is reused 200 times (2 per round for 100 rounds), so a waiting thread must be able to tell "my phase has been released" from "some later phase is collecting arrivals". The arrival count alone cannot do that, because the count is reset to 0 the moment the barrier opens and is immediately reused for the next phase.

Without a generation number, a waiter would have to wait on a condition such as waiting < parties. Suppose the last thread arrives, resets waiting to 0 and pulses. Before the other threads have re-acquired the lock, that fast thread can run its next phase and call the barrier again, making waiting = 1. When the earlier waiters finally wake up and re-check, they see waiting < parties, conclude that the barrier has not opened, and go back to sleep. Their wake-up has been lost and the program can hang; with other conditions a thread can instead slip through into the wrong phase.

With a generation number each thread remembers the generation it arrived in and waits while myGeneration == generation. The last arrival increments generation, and that change is permanent, so every thread of that phase gets through no matter how quickly the barrier is reused.

3. What did the no-barrier version produce? Was the wrong result the same in every run, and why does that program count as a data race even when its output looks right?

The no-barrier version was wrong in all 5 runs: 794, 794, 791, 756 and 782 cells did not match the reference (out of 800,000). The wrong result was not the same every time, because it depends on how the threads happened to be scheduled.

The numbers are close to 800 for a reason: errors start at the 4 boundaries between slices, where a worker reads a neighbour's cell from the wrong round, and then spread by one cell per round in each direction. After 100 rounds that is at most about 4 x 200 = 800 cells.

It is a data race because several threads read and write the same cells of current[] with no synchronization ordering those accesses, and at least one access is a write. That is a property of the program, not of one particular output. Even if a run happened to match the reference (for example if the threads stayed perfectly in step), the unsynchronized accesses would still be there and the next run could be wrong. As in Lab 04, a correct-looking output is not evidence that a program is free of races.

4. How did the time change with 8 workers compared with 4 on your machine? Why is the duration of a round determined by the slowest thread?

With 4 workers the average was 237.41 ms (174.70 to 278.47 ms); the single run with 8 workers took 324.05 ms, so 8 workers were slower even though the machine has 8 logical cores. The total work is the same (800,000 cells for 100 rounds), but with 8 workers there are 8 threads arriving at each of the 200 barrier calls, which means more lock acquisitions, more sleeping and waking, and more context

switches. The 8 logical cores are also probably hyper-threads of fewer physical cores and are shared with the operating system and other programs, so 8 compute-heavy threads do not get 8 full cores. The work per round per thread is small, so this overhead is larger than the gain.

A barrier releases nobody until the last thread arrives. So each phase lasts as long as the slowest worker takes, and faster workers simply wait. If one thread is delayed (preempted, on a busy core, or with a cache miss), every other thread loses that time too. With more threads it is more likely that at least one of them is delayed in any given round.

5. System.Threading.Barrier offers features that this hand-built barrier does not (for example a post-phase action and a changeable participant count). Explain why such features can be useful in round-based programs.

Post-phase action: System.Threading.Barrier can run a delegate once, by a single thread, after all participants have arrived and before any of them is released. In a round-based program that is the ideal place for a step that must happen exactly once per round while no worker is running: swapping the current and next buffers (instead of every worker copying its slice), incrementing the round counter, checking for convergence, or printing progress. It would also let this program use one barrier call per round instead of two.

Changeable participant count: AddParticipant and RemoveParticipant let workers join or leave between phases. That is useful when a worker finishes its part early, fails, or when the amount of work changes and threads are added or removed. With the hand-built barrier, a worker that stopped calling SignalAndWait() would leave the others waiting forever, because parties is fixed.

The library class also supports timeouts and cancellation, and reports a failed phase action through BarrierPostPhaseException, which helps a program avoid hanging when something goes wrong.

Section 6: Task 5 Lock-Free Stack with Compare-and-Swap (Optional Extension)

Source file: LockFreeStack/Program.cs

```
// Lab 05 - Task 5 (optional): Lock-Free Stack with Compare-and-Swap
// Push and Pop never take a lock. Each reads the head, prepares the change and
// installs it with Interlocked.CompareExchange (CAS); if another thread changed
// the head in between, the CAS fails and the operation is retried.
//
// Run:  dotnet run --no-build -c Release          (4 threads)
//       dotnet run --no-build -c Release -- 8     (8 threads)

using System;
using System.Diagnostics;
using System.Threading;

class Node
{
    public int Value;
    public Node? Next;

    public Node(int value)
    {
        Value = value;
    }
}

class LockFreeStack
{
    private Node? head = null;

    public void Push(int value)
    {
        Node node = new Node(value);

        while (true)
        {
            Node? oldHead = Volatile.Read(ref head);   // 1. read the current head
            node.Next = oldHead;                       // 2. link the new node in front of it
```

```
            // 3. install the new node only if head is still oldHead
            if (Interlocked.CompareExchange(ref head, node, oldHead) == oldHead)
                return;
            // otherwise another thread changed head: retry
        }
    }

    public bool TryPop(out int value)
    {
        while (true)
        {
            Node? oldHead = Volatile.Read(ref head);   // 1. read the current head
            if (oldHead == null)
            {
                value = 0;
                return false;                          // stack is empty
            }

            Node? newHead = oldHead.Next;              // 2. the node after it

            // 3. unlink oldHead only if head is still oldHead
            if (Interlocked.CompareExchange(ref head, newHead, oldHead) == oldHead)
            {
                value = oldHead.Value;
                return true;
            }
            // otherwise another thread changed head: retry
        }
    }

    public bool IsEmpty => Volatile.Read(ref head) == null;
}

class Program
{
    static int NumThreads = 4;
    const int NodesPerThread = 250_000;
    const int BatchSize = 100;                 // push a batch, then pop a batch

    static readonly LockFreeStack stack = new LockFreeStack();
    static int[] seen = Array.Empty<int>();    // seen[v] = how many times value v was popped

    static void Worker(int id)
    {
        int nextValue = id * NodesPerThread;   // every thread pushes its own range of values
        int pushed = 0;
        int popped = 0;

        while (popped < NodesPerThread)
        {
            for (int i = 0; i < BatchSize && pushed < NodesPerThread; i++)
            {
                stack.Push(nextValue++);
                pushed++;
            }

            for (int i = 0; i < BatchSize && popped < pushed; i++)
            {
                int value;
                while (!stack.TryPop(out value))
                {
                    Thread.Yield();            // temporarily empty: let the others push
                }
                Interlocked.Increment(ref seen[value]);
                popped++;
            }
        }
    }

    static void Main(string[] args)
    {
        if (args.Length > 0 && int.TryParse(args[0], out int n) && n > 0)
            NumThreads = n;

        int total = NumThreads * NodesPerThread;
        seen = new int[total];
```

```
    Thread[] threads = new Thread[NumThreads];

    Stopwatch sw = Stopwatch.StartNew();
    for (int t = 0; t < NumThreads; t++)
    {
        int id = t;                 // private copy for the lambda
        threads[t] = new Thread(() => Worker(id));
        threads[t].Start();
    }
    foreach (Thread th in threads) th.Join();
    sw.Stop();

    int lost = 0, duplicated = 0;
    for (int v = 0; v < total; v++)
    {
        if (seen[v] == 0) lost++;
        else if (seen[v] > 1) duplicated += seen[v] - 1;
    }

    Console.WriteLine($"Threads: {NumThreads}");
    Console.WriteLine($"Nodes (total): {total}");
    Console.WriteLine($"Lost: {lost}");
    Console.WriteLine($"Duplicated: {duplicated}");
    Console.WriteLine($"Stack empty: {(stack.IsEmpty ? "Y" : "N")}");
    Console.WriteLine($"Time: {sw.Elapsed.TotalMilliseconds:F2} ms");
  }
}
```

Run results (4 threads)

Run	Nodes (total)	Lost	Duplicated	Stack empty (Y/N)	Time (ms)
1	1,000,000	0	0	Y	188.17
2	1,000,000	0	0	Y	188.97
3	1,000,000	0	0	Y	176.48
4	1,000,000	0	0	Y	167.96
5	1,000,000	0	0	Y	126.62

Result with 8 threads: 2,000,000 nodes, Lost 0, Duplicated 0, Stack empty Y, Time 416.98 ms

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\LockFreeStack>dotnet run --no-build -c Release
Threads: 4
Nodes (total): 1000000
Lost: 0
Duplicated: 0
Stack empty: Y
Time: 188.17 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\LockFreeStack>dotnet run --no-build -c Release
Threads: 4
Nodes (total): 1000000
Lost: 0
Duplicated: 0
Stack empty: Y
Time: 188.97 ms
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\LockFreeStack>dotnet run --no-build -c Release
Threads: 4
Nodes (total): 1000000
Lost: 0
Duplicated: 0
Stack empty: Y
Time: 126.62 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab5_PDC\LockFreeStack>dotnet run --no-build -c Release -- 8
Threads: 8
Nodes (total): 2000000
Lost: 0
Duplicated: 0
Stack empty: Y
Time: 416.98 ms
```

Discussion Questions

1. Explain why the CAS retry loop never loses an element. Why is this stack lock-free but not wait-free?

Push reads head, links the new node in front of it, and then calls CompareExchange, which installs the new node only if head is still the value that was read. If another thread changed head in between, the CAS fails and changes nothing, and the loop repeats with the new head. So the stack is only ever modified by a successful CAS that was based on the current state; an update built on stale information is rejected instead of overwriting someone else's change. Pop works the same way. This is the opposite of the lost update in Lab 04, where a stale store was always written.

It is lock-free because whenever a CAS fails, it is because another thread's CAS succeeded, so the system as a whole always makes progress, and a thread that is paused never blocks the others (there is no lock to hold). It is not wait-free because there is no limit on how many times one particular thread may have to retry: in principle an unlucky thread could keep losing the race while others succeed.

2. Describe the ABA problem. Why can it not occur in this test, and why does .NET's garbage collector make it much less likely than in C or C++ even if nodes were reused?

ABA: a thread reads head = A and A.Next = B, and is then paused. Other threads pop A, pop B, and then push A back (the same node, or a new node at the same address). The first thread resumes, its CAS sees that head is A again and succeeds, and it sets head to B, although B is no longer in the stack. The CAS only compares the reference, so it cannot tell that the stack changed from A to something else and back to A.

It cannot happen in this test because every Push allocates a brand-new Node and a popped node is never pushed again, so a given reference appears at the head only once.

The garbage collector makes it much less likely in general: while the paused thread still holds a reference to A, the object cannot be collected, so its memory cannot be reused for a different node. A new node therefore cannot have the same reference as A. In C or C++ a popped node can be freed and malloc can return the same address for the next node, which is exactly how ABA usually arises. It could still happen in .NET if the program itself recycled node objects, for example from a pool.

3. Why is memory reclamation, a hard problem for lock-free structures in C, much simpler here?

In C, after a thread pops a node it cannot simply free it, because another thread may have read the same head pointer a moment earlier and be about to read node->next. Freeing it would be a use-after-free, and reusing the memory leads to ABA. Solving this needs extra machinery such as hazard pointers, epoch-based reclamation or reference counting, which is harder than the stack itself.

In .NET the garbage collector does this job. A node is reclaimed only when no thread holds a reference to it any more, including local variables of a thread that is in the middle of Pop. So a popped node is simply dropped: it stays valid for any thread still looking at it and is collected later. There is no manual free and therefore no use-after-free.

4. What exactly does the "no lost, no duplicated, stack empty" check prove, and what does it not prove about the correctness of the stack?

It proves that, in these runs, every value that was pushed was popped exactly once (the seen array has every entry equal to 1) and that nothing was left behind. With 4 threads that is 1,000,000 nodes in each of 5 runs, and 2,000,000 nodes with 8 threads, all with Lost 0, Duplicated 0 and Stack empty Y. So no element was lost or duplicated under the interleavings that actually occurred.

It does not prove that the stack is correct for every possible interleaving; like all testing, it only covers the schedules that happened. It does not check last-in, first-out order (linearizability) at all, only that the set of values is preserved. It does not exercise ABA, because nodes are never reused. And it says nothing about progress or fairness for individual threads. Correctness in general rests on the argument about the CAS loop, with the test as supporting evidence.

5. If time permits, compare the elapsed time with a mutex-protected stack doing the same work with 4 and with 8 threads. What do you observe, and does lock-free always mean faster?

A mutex-protected stack was not built or timed for this report, so there is no measured comparison. What was measured for the lock-free stack: 4 threads handled 1,000,000 nodes in 126.62 to 188.97 ms (average 169.64 ms), and 8 threads handled 2,000,000 nodes in 416.98 ms. That is about 0.17 microseconds per node with 4 threads and 0.21 with 8, so the cost per operation went up as threads were added rather than staying flat.

This is expected, and lock-free does not automatically mean faster. Every operation still has to change the same head reference, so its cache line moves between cores just as it does for a lock, and under heavy contention many CAS attempts fail and are repeated, which is wasted work. Each push also allocates a node. A simple lock with a very short critical section can match or beat a lock-free structure, especially with few threads. The real advantages of lock-free code are different: no thread can block the others by being paused while holding a lock, there are no deadlocks, and the system as a whole always makes progress.

Section 7: GitHub Repository

All source code for this lab is pushed to the lab5 branch of my repository:

https://github.com/fahad1084/-PDC_Labs/tree/lab5